

Lavorare con File di Testo nella CLI

Autore: Francesco Sardi

Data: 19 Febbraio 2026

Oggetto: Lavorare con File di Testo nella CLI - Preparazione test CyberOps

Executive Summary

In questo documento andremo a prendere familiarità con gli editor di testo (*grafici e da riga di comando*) in ambiente Linux, utilizzando la macchina virtuale Cyber Ops Workstation.

Obiettivi

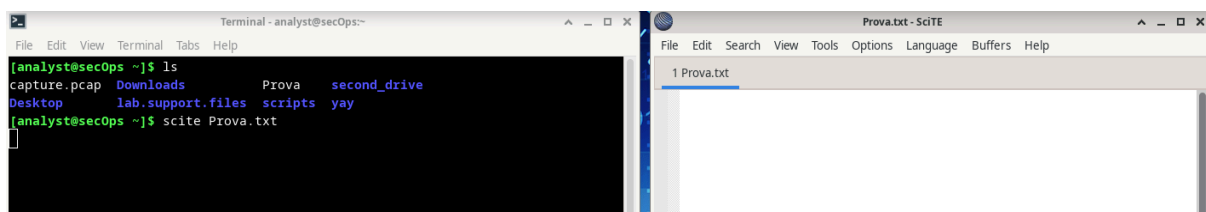
- **Parte 1:** Utilizzo di Editor di Testo Grafici
- **Parte 2:** Utilizzo di Editor di Testo da Riga di Comando
- **Parte 3:** Lavorare con i File di Configurazione

Work Delivery

Parte 1 & Fase 2: Editor di Testo Grafici

Come editor di testo grafico utilizzeremo **SciTE**, un software semplice, leggero e veloce preinstallato sulla nostra macchina, ideale per modifiche basilari senza l'ausilio di funzionalità troppo avanzate.

Abbiamo testato le funzioni base, come il procedimento di salvataggio e apertura di un file, completando l'operazione senza problemi. Successivamente, abbiamo aperto SciTE direttamente dal terminale verificando la presenza del file tramite il comando ls e lanciando l'editor con il comando `scite prova.txt`.



Fase 3: Editor da Riga di Comando

Per questa sezione utilizzeremo l'editor **nano**, preinstallato sui sistemi operativi Linux e di fondamentale importanza in caso di attacchi, accessi tramite SSH o assenza di interfaccia grafica.

Abbiamo aperto il file di prova digitando nel terminale: nano Prova.txt

Domanda: Quale carattere usa nano per rappresentare che una linea continua oltre i bordi dello schermo?

Risposta: Utilizza il carattere >.

Successivamente, tramite il comando `ls -a`, abbiamo visualizzato tutti i file della directory, inclusi quelli nascosti e di configurazione.

```
[analyst@secops ~]$ ls -a
.          Desktop      .idlerc      Prova        .vboxclient-clipboard-tty7-control.pid  .vboxclient-draganddrop-tty7-service.pid  .vim          .xsession-errors
..         bashrc          .dirc        lab.support.files  scripts    .vboxclient-clipboard-tty7-service.pid  .vboxclient-hostversion-tty7-control.pid  .viminfo      .xsession-errors.old
.bash_history  .cache        .dirc        .lessht         second_drive  .vboxclient-display-svga-x11-tty7-control.pid  .vboxclient-seamless-tty7-control.pid  .xauthority   .xsessionrc
.bash_logout  capture.pcap  .gnupg       .local          space.txt    .vboxclient-display-svga-x11-tty7-service.pid  .vboxclient-seamless-tty7-service.pid  .xinitrc      yay
.bash_profile  .config      .ICEauthority .mozilla        .ssh         .vboxclient-draganddrop-tty7-control.pid  .vboxclient-vmsvga-session-tty7-control.pid  .Xinitrc
[analyst@secops ~]$
```

Abbiamo poi utilizzato il comando `cat .bashrc` per visualizzarne il contenuto direttamente a schermo.

```
[analyst@secOps ~]$ cat .bashrc
export EDITOR=vim

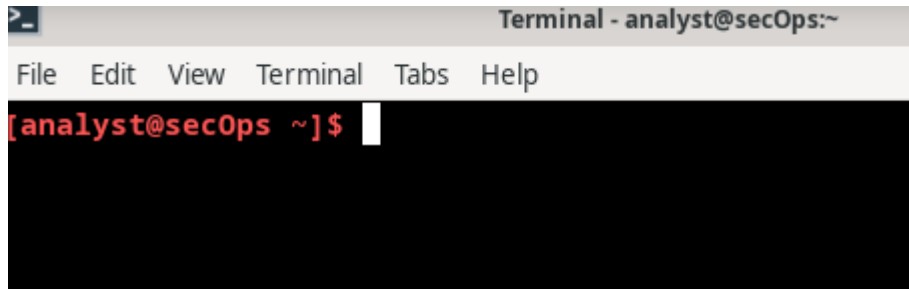
PS1='\[\e[1;32m\][\u@\h \W]\$ \[\e[0m\] '
alias ls="ls --color"
alias vi="vim"
[analyst@secOps ~]$
```

Domanda: Perché i file di configurazione delle applicazioni utente sono salvati nella directory home dell'utente e non sotto /etc con tutti gli altri file di configurazione a livello di sistema?

Risposta: Perché le configurazioni delle applicazioni sono soggettive e possono variare da utente a utente. Ogni utente ha la propria cartella Home con le proprie preferenze. Se così non fosse, e i file fossero globali in /etc, tutti gli utenti condividerebbero obbligatoriamente le stesse impostazioni per le stesse applicazioni.

Fase 4: Modifica Variabili d'Ambiente

Siamo andati a modificare il colore del prompt dei comandi (da verde a rosso). Per farlo, abbiamo modificato il file di configurazione .bashrc utilizzando SciTE, cambiando il codice colore da 32 (verde) a 31 (rosso).



Domanda: Anche la finestra del terminale che era già aperta ha cambiato colore da verde a rosso? Spiega.

Risposta: No, non è cambiata. Le impostazioni dei file come .bashrc vengono lette e caricate in memoria solo all'avvio di una nuova sessione o applicazione (o forzandone la rilettura con il comando source).

Fase 5: Modificare i File di Configurazione per i Servizi

In questa fase abbiamo lavorato sui file di configurazione dei servizi di sistema, nello specifico **Nginx**. Abbiamo aperto il file di configurazione di Nginx tramite nano.

```
GNU nano 8.5 /etc/nginx/custom_server.conf
1
2 #user html;
3 worker_processes 1;
4
5 #error_log logs/error.log;
6 #error_log logs/error.log notice;
7 #error_log logs/error.log info;
8
9 #pid logs/nginx.pid;
10
11
12 events {
13     worker_connections 1024;
14 }
15
16
17 http {
18     include mime.types;
19     default_type application/octet-stream;
20 }
```

Abbiamo modificato la porta di ascolto del servizio, cambiandola da 81 a 8080. Successivamente abbiamo provato ad avviare Nginx.

```
server {
    listen 8080;
    server_name localhost;
}
```

Domanda: A cosa si riferisce il messaggio di errore che compare avviando il servizio?

Risposta: Si riferisce al fatto che la porta 8080 è già in uso da un altro servizio o processo.

```
File Edit View Terminal Tabs Help
[analyst@secOps ~]$ sudo nginx -c /etc/nginx/custom_server.conf
[sudo] password for analyst:
2026/02/19 08:04:03 [emerg] 1455#1455: bind() to 0.0.0.0:8080 failed (98: Address already in use)
2026/02/19 08:04:03 [emerg] 1455#1455: bind() to 0.0.0.0:8080 failed (98: Address already in use)
2026/02/19 08:04:03 [emerg] 1455#1455: bind() to 0.0.0.0:8080 failed (98: Address already in use)
2026/02/19 08:04:03 [emerg] 1455#1455: bind() to 0.0.0.0:8080 failed (98: Address already in use)
2026/02/19 08:04:03 [emerg] 1455#1455: bind() to 0.0.0.0:8080 failed (98: Address already in use)
2026/02/19 08:04:03 [emerg] 1455#1455: still could not bind()
[analyst@secOps ~]$
```

Abbiamo quindi interrotto il servizio web Nginx, rendendo inaccessibile la pagina web.

Domanda: Puoi modificare il file `/etc/nginx/custom_server.conf` con SciTE? Descrivi il processo di seguito.

Risposta: Sì, posso farlo. Dal terminale apro il file con SciTE, scorro fino alle impostazioni che voglio alterare, applico le modifiche e salvo il file.

Parte Bonus: Familiarità con la Shell Linux

Comandi Base e Manuali

- **Elenca alcune sezioni incluse in una pagina man.** Ad esempio troviamo: *Name*, *Synopsis*, *Description*, *Examples*.
- **Qual è la funzione del comando cp?** Serve a copiare un file o una directory da un percorso di origine (source) a uno di destinazione.
- **Quale comando useresti per trovare maggiori informazioni sul comando pwd?** Userei `man pwd`. La funzione di `pwd` è quella di stampare a schermo il percorso completo della cartella corrente in cui ci si trova.

Navigazione delle Directory

- **Qual è la directory corrente?** /home/analyst
- **In quale cartella ti trovi ora?** cyops_folder3 (*cartella creata tramite mkdir ed in cui sono entrato tramite cd*).
- **Perché è successo?** Tramite cd ~ si torna direttamente alla cartella home dell'utente corrente (*nel nostro caso /home/analyst*).
- **Cosa succede eseguendo cd .?** Non succede nulla, poiché . rappresenta la directory corrente. Il comando ti lascia esattamente dove sei.
- **Cosa succede dopo aver digitato cd ..?** La directory corrente cambia spostandosi di un livello superiore. Si passa da *cyops_folder3* alla directory home dell'utente, indicata dal simbolo ~.
- **Quale sarebbe la directory corrente se eseguiessi il comando cd .. da [analyst@secOps ~]\$?** La risposta è /home. Il simbolo ~ rappresenta /home/analyst. Salendo di un livello con cd .., si arriva nella cartella /home.
- **Quale sarebbe la directory corrente se eseguiessi il comando cd .. da [analyst@secOps home]\$?** La risposta è /. Se ti trovi in /home e sali di un altro livello, arrivi alla base dell'intero file system.
- **Quale sarebbe la directory corrente se eseguiessi il comando cd .. da [analyst@secOps /]\$?** Rimarrebbe /. La directory radice è il livello più alto in assoluto del file system di Linux. Non essendoci nulla "sopra" di essa, il comando non ha alcun effetto.

Redirezione dell'Output

- **È previsto? Spiega.** Sì, è assolutamente previsto. L'operatore > serve proprio a redirigere l'output standard di un comando all'interno di un file.
- **Cosa è successo al file di testo? Spiega.** Il contenuto originale del file è stato completamente sovrascritto. L'uso di un singolo > cancella il contenuto precedente e lo sostituisce con il nuovo output.
- **Cosa è successo al file di testo? Spiega.** Il nuovo testo è stato accodato al file esistente. L'uso del doppio >> preserva il contenuto precedente e aggiunge le nuove informazioni in coda.

Visualizzazione File Nascosti

- **Quanti file vengono visualizzati?** L'output esatto non è visibile, ma vengono elencati solo i file e le directory standard, escludendo quelli nascosti.
- **Quanti file in più vengono visualizzati rispetto a prima? Spiega.** Vengono mostrati in più tutti i file di configurazione o di sistema che iniziano con un punto oltre alle directory speciali.
- **È possibile nascondere intere directory aggiungendo un punto prima del loro nome? Ci sono directory nascoste nell'output di ls -la sopra?**

Sì, è possibile nascondere le directory antepoendo un punto al nome. Nell'output compaiono sicuramente le directory nascoste . e ...

Manipolazione File

Copia e Spostamento

- **Quali sono i file sorgente e destinazione? (Usa percorsi completi)**
 - Sorgente: /home/analyst/some_text_file.txt .
 - Destinazione: /home/analyst/cyops_folder2/
- **Quale comando hai usato per completare l'attività?** Il comando usato per riportare il file nella directory home è stato: mv cyops_folder2/some_text_file.txt .

Riflessione Finale

Quali sono i vantaggi dell'utilizzo della riga di comando Linux (CLI)?

- **Velocità ed Efficienza:** Per gli utenti esperti, digitare comandi è molto più rapido che navigare all'interno di un'interfaccia grafica con il mouse.
- **Automazione:** I comandi possono essere raggruppati in script bash per eseguire task complessi o ripetitivi in modo del tutto automatico.
- **Basso consumo di risorse:** Un'interfaccia a riga di comando richiede una quantità nettamente inferiore di memoria e potenza di calcolo rispetto a un'interfaccia grafica .
- **Gestione Remota:** È il metodo standard, più sicuro e più affidabile per amministrare server Linux da remoto tramite protocolli come SSH, che richiedono pochissima banda di rete.